



Data Analytics mit SQL & Python

VON DATEN ZUR ENTSCHEIDUNG

DR. CHIAO-YA CHANG
CHIAOYA.CHANG.TW@GMAIL.COM

Tag 3 – Grundlagen Python



Ablauf

Tag 3 – Python & Datenanalyse		
09:00–09:30	Wiederholung & Einstieg	
09:30–10:30	Python-Grundlagen	Datentypen, Variablen, Schleifen, Funktionen
10:30–11:30	Arbeiten mit Daten (Pandas)	Daten laden, filtern, berechnen
11:30–12:10	 Mittagspause	—
12:10–14:00	Datenvisualisierung	Matplotlib / Seaborn: Diagramme & Insights
14:00–14:20	 Pause	—
14:20–15:40	Mini-Projekt: Explorative Analyse mit Python	Gruppenarbeit mit realem Datensatz
15:40–16:00	Ergebnisse & Diskussion	Präsentation, Reflexion, Q&A





Tag 3 09:00–09:30

Wiederholung & Einstieg

- SQL: JOINS, GROUP BY, HAVING, Subqueries
- Was ist der Unterschied zwischen SQL und Python?
- Verbindung: SQL fragt Daten ab, Python analysiert und visualisiert Daten

Wiederholung & Einstieg

30-minütige Übung mit Fitness-Daten

Die Teilnehmenden sollten jedoch:

- **Google Colab öffnen** und die Datei „Day_3_Data_Analytics_SQL_Python_Exercises.ipynb“ laden
- **Teil A zur Wiederholung von Tag 2 bearbeiten**

Ziel:

- Aktivierung des Vorwissens
- Einstieg in die Arbeit mit realen Daten





Tag 3 09:30–10:30

Python-Grundlagen

- Datentypen
- Variablen
- Schleifen
- Funktionen

Aufgabe: Python Installation

- Link: <https://www.python.org/>
- Jupyter Notebook file: file_name.**ipynb**
- Python file: file_name.**py**
- **Hausaufgaben:** Python installation mit privaten PC/Laptop
- **Kursmaterial (GitHub)**





Tag 3 10:30–11:30

Arbeiten mit Daten (Pandas)

- Daten laden, filtern, berechnen

Was ist Pandas?

Warum Pandas verwenden?

Pandas ist eine Python-Bibliothek, die zur Arbeit mit **Datensätzen** verwendet wird.

Pandas bezieht sich sowohl auf „**Panel Data**“ als auch auf „**Python Data Analysis**“.

Funktionen zum:

- Analysieren, bereinigen und erkunden von Daten
- Verändern und Bearbeiten von Daten

Pandas ermöglicht es uns:

- große Datenmengen zu analysieren
- Schlussfolgerungen auf Basis statistischer Methoden zu ziehen



DataFrame verstehen?

Ein DataFrame ist **wie eine Excel-Tabelle in Python**:

Zeilen (rows)	Index	Name	Abteilung	Umsatz
	0	Anna M.	Marketing	12.500 €
	1	Ben K.	Vertrieb	18.200 €
	2	Clara S.	HR	9.800 €
	3	David L.	IT	21.000 €
Spalten (columns)				

Wichtige Befehle:

```
df.shape
```

(4, 4) → 4 Zeilen, 4 Spalten

```
df.columns
```

["Name", "Abteilung", "Umsatz"]

```
df["Umsatz"]
```

→ eine Spalte auswählen

```
df.iloc[0]
```

→ erste Zeile auswählen

Daten laden mit Pandas

Schritt 1: Bibliotheken importieren

```
import pandas as pd
import matplotlib.pyplot as plt
```

Schritt 2: CSV-Datei einlesen

```
df = pd.read_csv("fitness_daten.csv")
print(df.shape)      # z.B. (1000, 8)
print(df.dtypes)     # Spaltentypen anzeigen
```

Schritt 3: Ersten Überblick verschaffen

```
df.head(5)           # erste 5 Zeilen
df.tail(3)            # letzte 3 Zeilen
df.sample(5)          # 5 zufällige Zeilen
```

Weitere Formate:

```
pd.read_excel("datei.xlsx")
```

```
pd.read_json("datei.json")
```

```
pd.read_sql(query, conn)
```

Tipp: Datei muss im gleichen Ordner wie das Notebook liegen — oder den vollständigen Pfad angeben, z. B.
"/content/drive/MyDrive/daten.csv"

Daten bereinigen

Echte Daten sind selten perfekt — Bereinigung ist der wichtigste Schritt!

1. Fehlende Werte erkennen

```
df.isna().sum()  
# zeigt Anzahl NaN pro Spalte
```

2. Fehlende Werte behandeln

```
df.dropna()           # Zeilen löschen  
df.fillna(0)          # mit 0 auffüllen  
df["Alter"].fillna(df["Alter"].mean())
```

3. Duplikate entfernen

```
df.duplicated().sum()  # Anzahl  
df.drop_duplicates()   # entfernen
```

4. Datentypen korrigieren

```
df["Datum"] = pd.to_datetime(df["Datum"])  
df["Preis"] = df["Preis"].astype(float)
```

Filtern & Gruppieren

Daten filtern

```
# Nur Zeilen wo Umsatz > 10000
df[df["Umsatz"] > 10000]

# Mehrere Bedingungen
df[(df["Abteilung"] == "Vertrieb") &
    (df["Umsatz"] > 15000)]

# Bestimmte Spalten auswählen
df[["Name", "Umsatz"]]
```

Merke: `groupby()` ist das Python-Äquivalent zu SQL GROUP BY — gleiche Logik, andere Syntax!

Daten gruppieren

```
# Durchschnittsumsatz je Abteilung
df.groupby("Abteilung")["Umsatz"].mean()

# Mehrere Aggregationen
df.groupby("Abteilung").agg({"Umsatz": ["mean",
    "sum", "count"]})

# Sortieren
df.sort_values("Umsatz", ascending=False)
```

Ergebnis-Beispiel:

Abteilung	Ø Umsatz
Marketing	14.200 €
Vertrieb	19.800 €
HR	8.500 €

11:30–12:10 🍴 Mittagspause



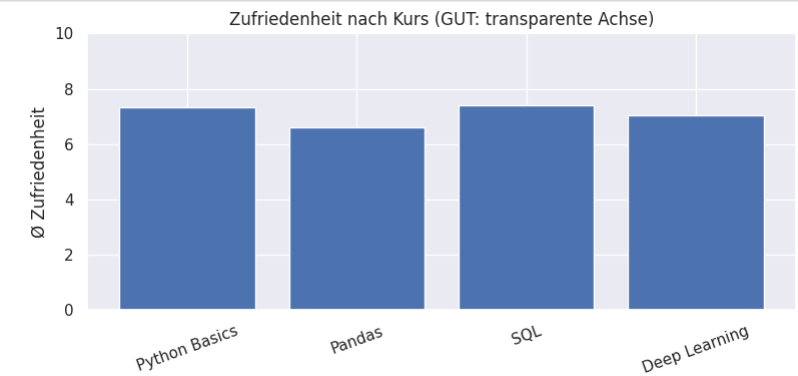
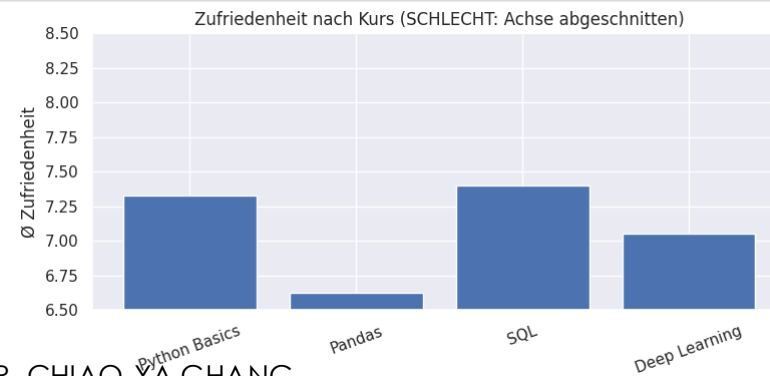
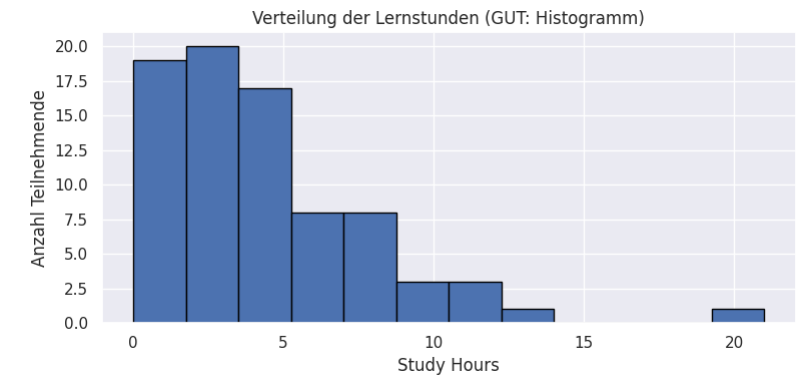
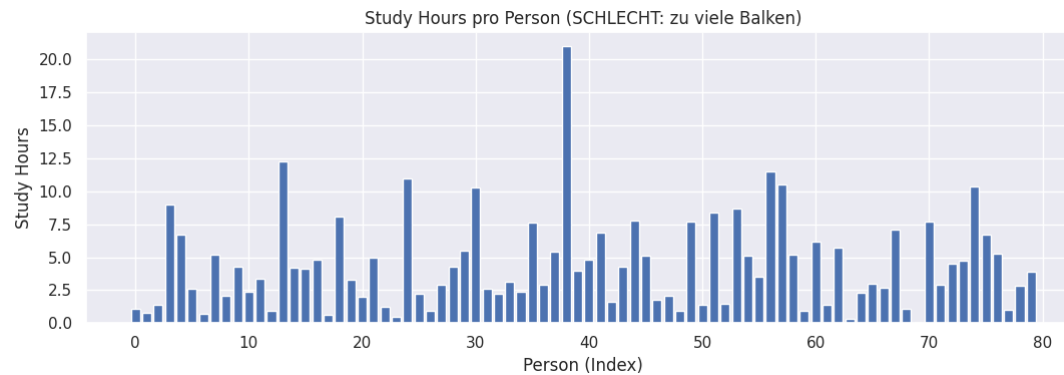
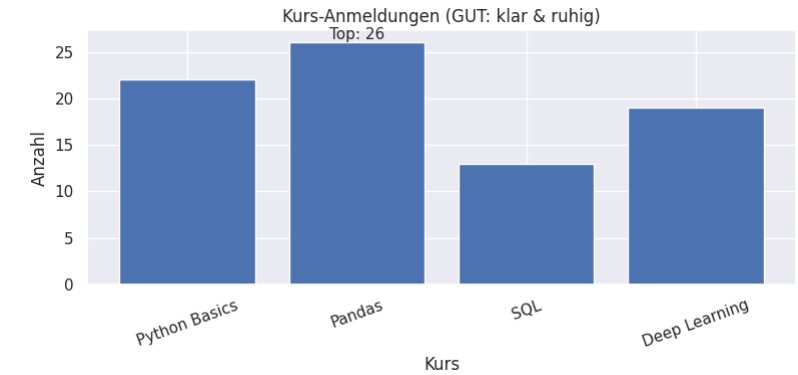
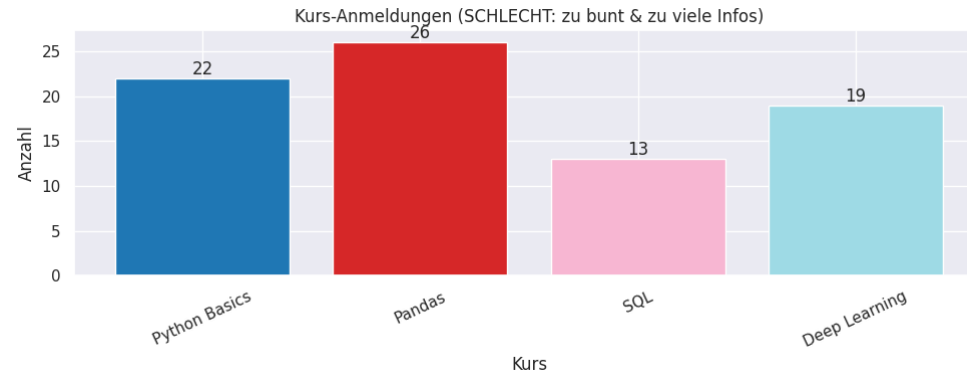


Tag 3 12:10–14:00

Datenvisualisierung

- Matplotlib / Seaborn
- Diagramme & Insights

Gute vs. schlechte Visualisierung



14:00–14:20 ☕ Pause





Tag 3 14:20–15:40

Mini-Projekt: Explorative Analyse mit Python

- Gruppenarbeit mit realem Datensatz



Tag 3 15:40–16:00

Ergebnisse & Diskussion

- Präsentation
- Reflexion
- Q&A